Personal Project Report

Mason Jones

The goal of my project was to create new mathematical operators and train a neural network on them to see if it can answer questions using them. This was intended to be used to help in reverse engineering functions redefined as operands, but the results instead turned into an analysis of how a model like this loses accuracy when complexity is added. The ability to reverse engineer undefined functions would be useful for various fields to be able to create formulas to calculate whatever measurable phenomenon where a formula wasn't available. The type of network used was a LSTM to avoid the short term memory problems that RNNs have as the dataset contains a large amount of equations and to attempt to avoid a vanishing or exploding gradient. A LSTM is able to learn algorithms and generalize the answer like a learnable language, which should be enough to create approximations for unknown operators.

I created four different made up operators and set up three different difficulty levels to train the AI off of. These difficulties alter the number of steps to each equation, and the magnitude of the numbers used. The four new operators are as follows:

& : a / (2 * b), absolute last in order of operations

$: b * (ceiling(b / a)), same priority as addition and subtraction

: a + ((3 + a) * b), same priority as multiplication and division

@ : -a - (b * 5), same priority as exponents

The difficulties are easy, medium, and hard, with each changing the number of numbers, operators, and range the numbers can be.

Easy allows for two numbers, one operator, and a range from -9 to 9. Examples of easy equations are:

8 & 2 = 2

4 / 2 = 2

-9 - 5 = -14

Medium allows for three numbers, two operators, and a range from -49 to 49 as well as adding the ability to use the modulo operator. Examples of medium equations are:

10 + 20 + 30 = 60

-5 * 5 / -5 = 5

40 / 4 % 5 = 0

Hard allows for five numbers, four operators, and a range from -99 to 99 and still has access to modulo. Examples of hard equations are:

-96 - 14 + -8 - -7 - -12 = -99

2 * 80 / 8 - 34 + 5 = -9

1 * 2 * 3 * 4 * 5 = 120

A LSTM is trained on 100000 equations at each difficulty at each total number of new operators (0 - 4) to create 15 total models to see the varying accuracy the operators and difficulty have. The hyperparameters were consistent between each configuration with 100000 equations, 250 epochs, 32 batch size, 128 embed size, 512 hidden size, and a learning rate of 0.0005. The overall accuracy is gained by asking the answer to 500 equations of the same difficulty and operators and averaging how close the predicted answer versus the real answer is.

The baseline for each difficulty with no new operators is as follows:

Easy, 0 operator: 99.895% accurate

Medium, 0 operator: 70.082% accurate

Hard, 0 operator: 3.334% accurate

These already paint the picture of how unreliable the model becomes with complexity, but seeing the change to these baselines when new operators are added will be insightful anyway.

The results when the new operators were added is as follows:

Easy, 1 operator: 99.673% accurate

Easy, 2 operator: 99.549% accurate

Easy, 3 operator: 99.698% accurate

Easy, 4 operator: 99.134% accurate

Medium, 1 operator: 48.840% accurate

Medium, 2 operator: 42.505% accurate

Medium, 3 operator: 40.525% accurate

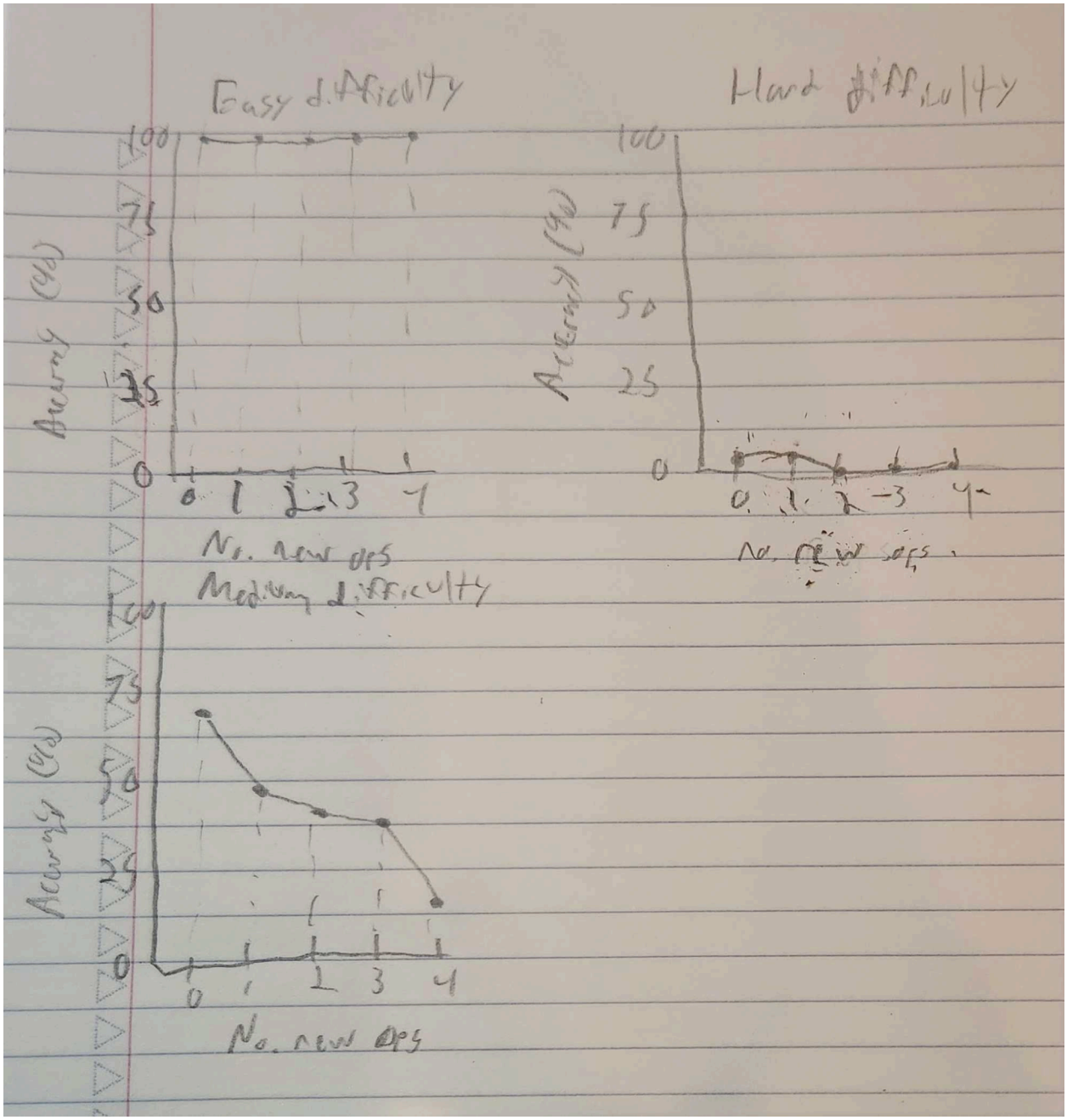
Medium, 4 operator: 14.085% accurate

Hard, 1 operator: 4.336% accurate

Hard, 2 operator: 0.395% accurate

Hard, 3 operator: 0.258% accurate

Hard, 4 operator: 0.439% accurate



These results show that as complexity increases, accuracy decreases. Medium difficulty was the only one in a goldilocks zone to meaningfully see the impact new operators made, with each new operator decreasing accuracy by significant amounts. The more complex the equations and the amount of new operators decrease the accuracy by so much, anything outside of easy is

unreliable and not useful for reverse engineering the operators. This shows how expansive of a question this really is, as the amount of possible equations and possible answers skyrocket the more range and complexity equations are allowed to use, very quickly becoming too much for a model of this magnitude. To add further insult to this exercise, I also tested each four operator model on a test set with an unlocked range the numbers could be to simulate new equations the model has never seen before which are as follows:

Easy, 4 operators, range expanded (-100, 100): 14.186% accurate

Medium, 4 operators, range expanded (-150, 150): 13.272% accurate

Hard, 4 operators, range expanded (-200, 200): 1.009% accurate

These are far too low to be used. It's easy to see why this result happens, as the numbers in these equations are not in the dataset it trained off of, so the tokens representing these numbers are meaningless, so the model defaults to the mean of the training set, which can be seen with medium reporting the answer to every equation with new numbers as 143. The naive solution to this is to include as many numbers as possible in the dataset so no token is unrecognized, but you can see how that turns out with the results of the hard difficulty. There is a hard limit with this setup, and its extremely basic equations with small numbers. Within this limited range of low complexity, it does a surprisingly good job and is very accurate at approximating these operators in the rest of the easy options. A potential improvement would be to have more computational hardware (or time) to handle more equations to get as many different tokens accounted for as possible.

There aren't any ethical considerations for the use of an AI like this, as the data was sourced from a program I made, nor are random equations exactly copywritable. There was no AI tool usage in the creation of this assignment.

In conclusion, a LSTM trained at this magnitude cannot be used to help decode and answer equations with made up operators unless the equations in question are very simple, in which you’re probably better off reverse engineering the operators by hand. This is due to the explosive number of possibilities higher degree equations can make as well as unfamiliar numbers defaulting the model to the average. But even if silly, it can predict the answer to extremely simple equations.